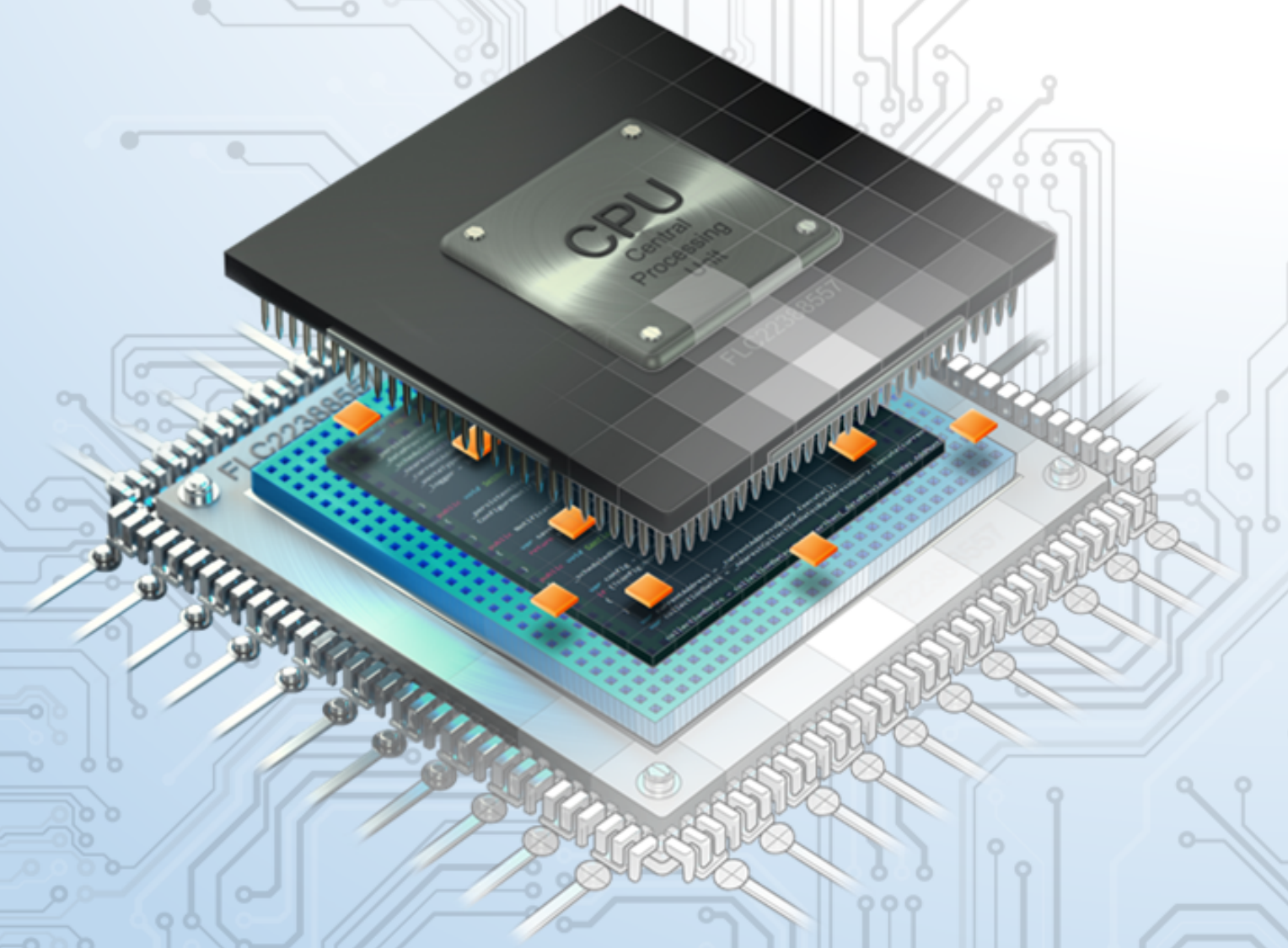
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
COMPUTER ENGINEERING

Microcontroller



Dr. Le Trong Nhan

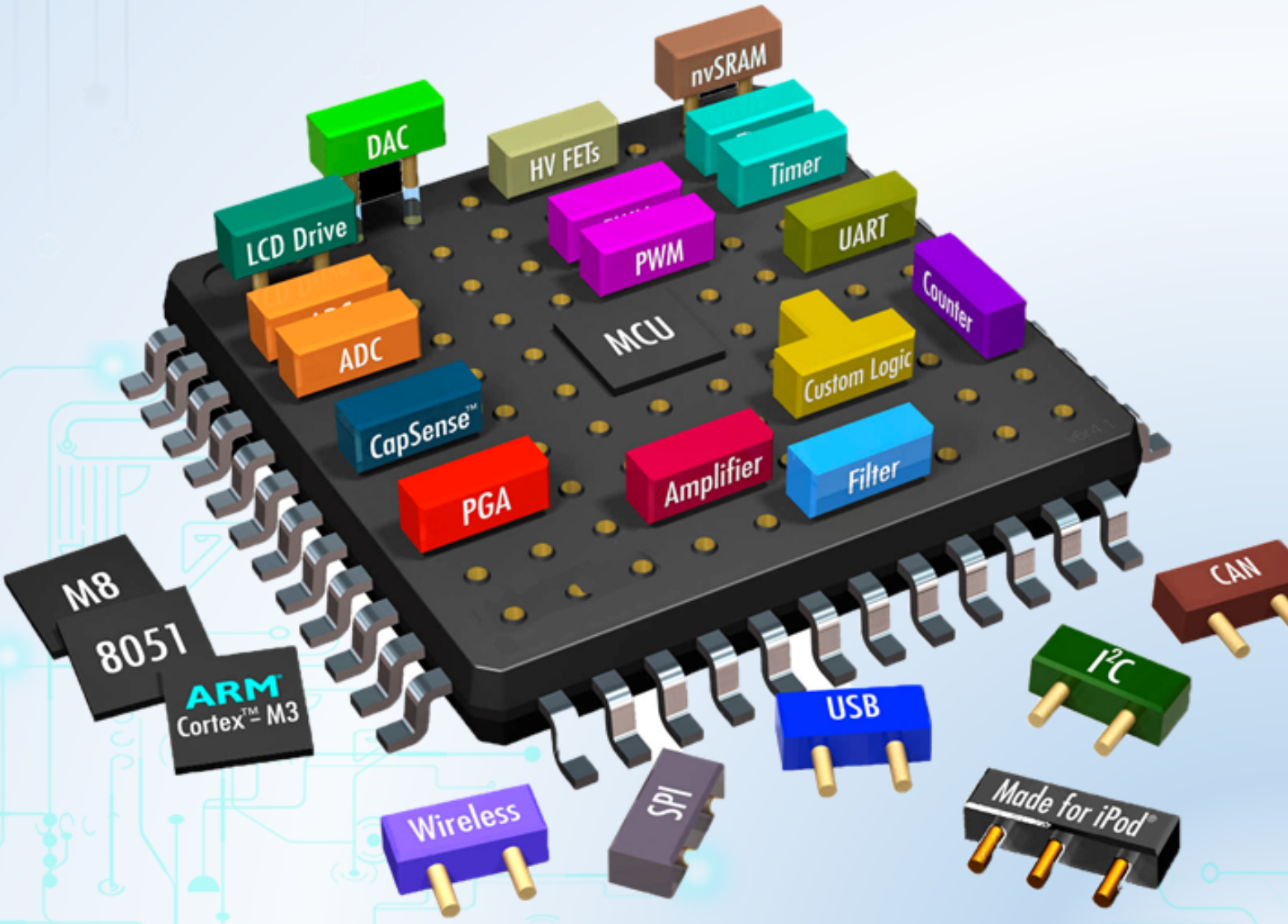
Mục lục

Chapter 4. A cooperative scheduler **5**

1 Objectives 6
2 Background 6
2.1 Super loop and timer interrupts 6
2.2 What is a scheduler? 6
2.3 Function pointers 7
3 Problem 7
4 Hardware and project configuration 7
4.1 STM32CubeMX configuration 7
4.2 Proteus schematic 8
5 Scheduler design 9
5.1 Data structure: a delta queue 9
5.2 The functions 10
5.3 Order of callbacks 10
5.4 Shared data between the ISR and the main loop 10
5.5 Source code 10
6 Application and demonstration code 16
7 Verification 18
7.1 Host unit tests 18
7.2 Simulation result 19
8 Problems found during simulation 22
8.1 Firmware stopped in the first version 22
8.2 Timer period in Proteus 22
8.3 Random Proteus exception 23
9 Conclusion 24

CHƯƠNG 4

A cooperative scheduler



1 Objectives

The aim of this lab is to design and implement a cooperative scheduler that accurately provides timeouts and triggers activities. In this lab, I

- add a separate C module (`scheduler.c`/`scheduler.h`) for the scheduler,
- drive the scheduler from a single 10 ms timer interrupt (TIM2),
- modify the main loop so that it only dispatches due tasks,
- run five periodic tasks (0.5 s, 1 s, 1.5 s, 2 s, 2.5 s) together with one-shot timeouts, and print a timestamp over UART every time a callback runs.

2 Background

2.1 Super loop and timer interrupts

The simplest embedded program is a *super loop*: initialise once, then call the task forever. It needs almost no memory, but it cannot run a task at a precise interval. Adding a software delay after the task (for example `X(); Delay_190ms();` to get a 200 ms period) only works if the duration of `X()` is known and never changes, which is rarely true once the task interacts with the outside world or the compiler settings change.

A better approach is to use a timer interrupt as the time base: the hardware timer overflows at a fixed rate independent of how long the tasks take, and the interrupt service routine (ISR) records that time has passed.

2.2 What is a scheduler?

A scheduler can be viewed in two ways:

- as a very small operating system that calls tasks periodically or once after a delay;
- as a single timer ISR shared between many tasks, so only one timer has to be configured no matter whether there are one, ten or a hundred tasks.

In a *co-operative* scheduler each task runs to completion and then returns control to the scheduler. Tasks never pre-empt each other, so no locking is needed between tasks, the behaviour is predictable, and the scheduler is written entirely in C as part of the application. The price is that every task must be short: a task that blocks delays all the others.

2.3 Function pointers

The scheduler stores tasks as *function pointers*. Just as a pointer can hold the address of a variable, a function pointer holds the address of the code of a function and can be used to call it:

```
void (*pTask)(void);   /* pointer to a function with no
   arguments and no return value */
pTask = Led_Display;   /* store the address of the function */
(*pTask)();            /* call it; pTask() is equivalent */
```

Program 4.1: Calling a function through a pointer

`SCH_Add_Task` receives such a pointer and the dispatcher later calls it when the task is due.

3 Problem

The system must provide at least the following functions:

- `void SCH_Update(void)`: updates the remaining time of the tasks in the queue. It is called from the timer interrupt, every 10 ms.
- `void SCH_Dispatch_Tasks(void)`: takes the due tasks from the queue and runs them.
- `uint32_t SCH_Add_Task(void (*pFunction)(), uint32_t DELAY, uint32_t PERIOD)`: adds a task and returns an ID for it.
- `uint8_t SCH_Delete_Task(uint32_t taskID)`: deletes a task by its ID.

The main program must have five tasks running periodically at 0.5 s, 1 s, 1.5 s, 2 s and 2.5 s. The demonstration must show a regular 10 ms tick, a 10 ms timeout that prints the value of `get_time`, several timeouts registered at the same time, and one-shot tasks registered before the main loop that print the timestamp when they expire. Timestamps must be accurate to 10 ms. The following designs are considered unsatisfactory: supporting only one timeout at a time, delivering callbacks in the wrong order, and O(n) searches in `SCH_Update`.

4 Hardware and project configuration

4.1 STM32CubeMX configuration

The project `LAB4` is generated for the STM32F103C6 with the settings in Table 4.1. The board of Lab 3 (four 7-segment digits, two sets of traffic lights and three buttons) is kept, so the same schematic can be reused later. Two enable lines of the 7-segment display were moved to free PA9/PA10 for USART1, and five LEDs were added for the five periodic tasks (Table 4.2).

Setting	Value
Clock	HSI 8 MHz, no PLL; SYSCLK = APB1 = APB2 = 8 MHz
SYS Debug	No Debug (frees PB3, PB4 and PA15 for GPIO)
TIM2	Prescaler 79, Period (ARR) 999, global interrupt enabled
USART1	Asynchronous, 115200 baud, 8N1
GPIO outputs	Push-pull, initial level High (all LEDs are active low)
GPIO inputs	PB0–PB2, no pull (external pull-up resistors)

Bảng 4.1: CubeMX settings

The timer interrupt period is

$$T = \frac{(PSC+1)(ARR+1)}{f_{TIM}} = \frac{80 \times 1000}{8\,000\,000\ \text{Hz}} = 10\ \text{ms}.$$

Section 8.2 explains why a large ARR (999) is used instead of the equivalent PSC 7999 / ARR 9.

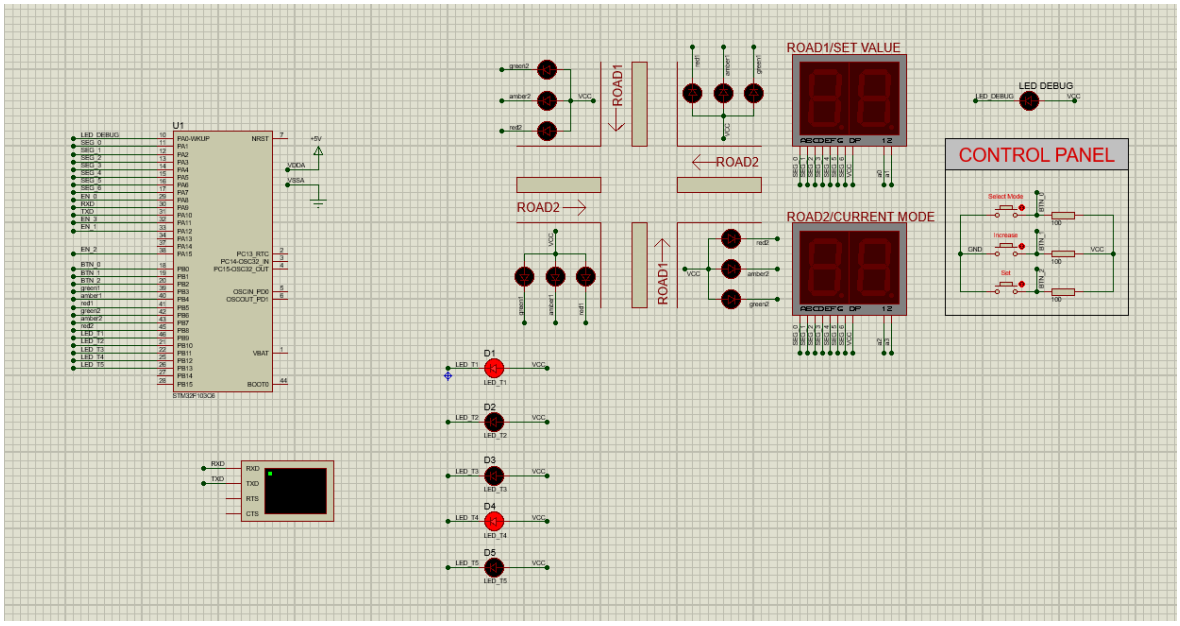
115200 baud is used instead of 9600 because the 10 ms demo task prints a line of about 25 characters every tick: at 9600 baud such a line already takes about 26 ms to send, longer than the tick, while at 115200 baud it takes about 2.2 ms.

Function	Pin	Label
Debug LED	PA0	`LED_DEBUG`
7-segment segments a–g	PA1–PA7	`LED7SEG_a` ... `LED7SEG_g`
7-segment enables	PA8, PA12, PA15, PA11	`EN_0` ... `EN_3`
UART TX / RX	PA9 / PA10	USART1
Buttons	PB0, PB1, PB2	`BUTTON_0` ... `BUTTON_2`
Road 1 green / amber / red	PB3 / PB4 / PB5	`GREEN1`, `AMBER1`, `RED1`
Road 2 green / amber / red	PB6 / PB7 / PB8	`GREEN2`, `AMBER2`, `RED2`
Task LEDs 0.5 / 1 / 1.5 / 2 / 2.5 s	PB9–PB13	`LED_T1` ... `LED_T5`

Bảng 4.2: Pin assignment

4.2 Proteus schematic

Figure 4.1 shows the schematic. Each task LED is connected with its cathode to the MCU pin and its anode to VCC, so writing 0 turns it on. A Virtual Terminal is connected to USART1 (its RXD to PA9, the TX pin of the MCU) and set to 115200 baud, 8 data bits, no parity, 1 stop bit.



Hình 4.1: *Proteus schematic: STM32F103C6, Lab 3 board, task LEDs D1–D5 and Virtual Terminal*

5 Scheduler design

5.1 Data structure: a delta queue

If every task stored its own remaining time, `SCH_Update` would have to decrement all of them in every tick, which is O(n). Instead, the tasks are kept in a *delta queue*: a linked list sorted by due time, where each node stores only the number of ticks *after the node before it*. The head stores the ticks until it is due. Decrementing the head therefore moves the whole queue one tick closer, so `SCH_Update` is O(1).

For example, tasks due in 50, 100 and 150 ticks are stored as $50 \rightarrow 50 \rightarrow 50$. A new task due in 120 ticks is inserted after the second node with delta $120 - 100 = 20$, and the delta of the node behind it becomes $50 - 20 = 30$: $50 \rightarrow 50 \rightarrow 20 \rightarrow 30$.

The list is built on a static array of `SCH_MAX_TASKS` = 16 slots (no dynamic memory). Each slot holds:

- `pTask`: the function pointer;
- `delay`: ticks after the previous node (head: ticks from now);
- `period`: period in ticks, 0 for a one-shot task;
- `id`: the ID returned to the user;
- `seq`: registration order, used to order tasks due at the same tick;
- `next`, `used`: list link and allocation flag.

5.2 The functions

- `SCH_Init`: marks all slots free and resets the time.
- `SCH_Update` (TIM2 ISR): counts the time and decrements only the head. If the head is already due but the main loop has not run it yet (because another task is still running), the tick is stored in `late_ticks` and later subtracted from the tasks behind it, so a long task does not shift the timing of the others.
- `SCH_Add_Task`: takes a free slot, converts DELAY and PERIOD from ms to ticks (rounded down) and inserts the slot into the queue. It returns `(generation << 8) | slot`; the generation counter of a slot is increased every time the slot is reused, so an old ID can never delete a newer task that happens to use the same slot. It returns `SCH_NO_TASK_ID` when the queue is full.
- `SCH_Delete_Task`: checks that the ID still matches the slot, unlinks the node and adds its delta to the next node so the following tasks keep their due times. It returns 1 on success and 0 for an unknown or already deleted ID.
- `SCH_Dispatch_Tasks` (main loop): while the head is due, removes it, puts a periodic task back with `delay = period`, frees a one-shot task, then calls the task. When nothing is due it executes `__WFI()` to sleep until the next interrupt.
- `SCH_Get_Time`: the time since `SCH_Init` in ms (the `get_time` of the problem statement), with a resolution of one tick.

5.3 Order of callbacks

Tasks that are due at the same tick run in the order in which they were registered with `SCH_Add_Task`. When a node is inserted, it is placed after all nodes with an earlier due time and after the nodes with the *same* due time but a smaller `seq`. This also holds after a periodic task is put back into the queue, so at 1000 ms T1 always runs before T2.

5.4 Shared data between the ISR and the main loop

The ISR changes the head of the list while the main loop inserts and removes nodes. Every list operation in the main loop is therefore done with interrupts disabled (saving and restoring PRIMASK), and the task itself is called after interrupts are enabled again, so a long task never blocks the timer interrupt. Variables written by the ISR are `volatile`.

5.5 Source code

```
/*
 * scheduler.h
 *
```

```
 * Cooperative scheduler for Lab 4.
 * Tasks are kept in a delta queue sorted by due time: each entry stores
 * the number of ticks after the entry before it. SCH_Update() therefore
 * only touches the head of the queue (O(1)) inside the timer ISR.
 */

#ifndef INC_SCHEDULER_H_
#define INC_SCHEDULER_H_

#include <stdint.h>

#define SCH_TICK_MS     10u   /* TIM2 interrupt period */
#define SCH_MAX_TASKS   16u
#define SCH_NO_TASK_ID  0xFFFFFFFFu

void     SCH_Init(void);
void     SCH_Update(void);           /* call from the TIM2 interrupt */
void     SCH_Dispatch_Tasks(void);   /* call from the main loop */

/* DELAY and PERIOD are in ms, rounded down to whole ticks.
 * PERIOD below SCH_TICK_MS (e.g. 0) runs the task once.
 * Returns SCH_NO_TASK_ID when the queue is full. */
uint32_t SCH_Add_Task(void (*pFunction)(void), uint32_t DELAY, uint32_t PERIOD);

/* Returns 1 when the task was removed, 0 when the ID is unknown. */
uint8_t  SCH_Delete_Task(uint32_t taskID);

/* Time since SCH_Init in ms, accurate to one tick. */
uint32_t SCH_Get_Time(void);

#endif /* INC_SCHEDULER_H_ */
```

Program 4.2: scheduler.h

```
/*
 * scheduler.c
 *
 * Delta-queue cooperative scheduler.
 *
 * - The queue is a singly linked list over a static pool, sorted by due
 *   time. delay of the head = ticks until it is due; delay of every other
 *   node = ticks after the node before it.
 * - SCH_Update (ISR) only decrements the head: O(1). If the head is already
```

```
 *   due and the main loop has not run it yet, the tick is counted in
 *   late_ticks so no time is lost for the tasks behind it.
 * - SCH_Dispatch_Tasks (main loop) removes due tasks one at a time, puts
 *   periodic tasks back with delay = period, then runs the task with
 *   interrupts enabled. Tasks due at the same tick run in registration
 *   order (SCH_Add_Task call order), including periodic tasks put back.
 * - A task ID is (generation << 8) | slot, so a stale ID cannot delete a
 *   task that later reused the same slot.
 */

#include "scheduler.h"
#include "main.h"

#define SCH_NONE 0xFFu

typedef struct {
    void     (*pTask)(void);
    uint32_t delay;    /* ticks after the previous node (head: ticks from now) */
    uint32_t period;   /* ticks, 0 = one-shot */
    uint32_t id;
    uint32_t seq;      /* registration order, breaks ties between equal due times */
    uint8_t  next;
    uint8_t  used;
} sTask;

static sTask    tasks[SCH_MAX_TASKS];
static uint8_t  head = SCH_NONE;
static uint8_t  generation[SCH_MAX_TASKS];
static uint32_t next_seq;
static volatile uint32_t late_ticks;
static volatile uint32_t time_ticks;

static uint32_t lock(void) {
    uint32_t primask = __get_PRIMASK();
    __disable_irq();
    return primask;
}

static void unlock(uint32_t primask) {
    __set_PRIMASK(primask);
}

/* Link a used slot into the queue. Caller holds the lock. */
static void insert(uint8_t slot, uint32_t delay) {
```

```
54      uint8_t prev = SCH_NONE;
55      uint8_t cur = head;
56
57      /* Tasks due at the same tick stay in registration order, also after a
58       * periodic task is put back, so T1 always runs before T2 at 1000 ms. */
59      while (cur != SCH_NONE && (tasks[cur].delay < delay
60              || (tasks[cur].delay == delay && tasks[cur].seq < tasks[slot].seq))) {
61          delay -= tasks[cur].delay;
62          prev = cur;
63          cur = tasks[cur].next;
64      }
65
66      tasks[slot].delay = delay;
67      tasks[slot].next = cur;
68      if (cur != SCH_NONE) {
69          tasks[cur].delay -= delay;
70      }
71      if (prev == SCH_NONE) {
72          head = slot;
73      } else {
74          tasks[prev].next = slot;
75      }
76  }
77
78  /* Hand ticks that arrived while the head was due to the tasks behind it. */
79  static void apply_late_ticks(void) {
80      while (late_ticks > 0 && head != SCH_NONE && tasks[head].delay > 0) {
81          uint32_t step = late_ticks < tasks[head].delay ? late_ticks : tasks[head].delay;
82          tasks[head].delay -= step;
83          late_ticks -= step;
84      }
85      if (head == SCH_NONE) {
86          late_ticks = 0;
87      }
88  }
89
90  void SCH_Init(void) {
91      uint32_t primask = lock();
92      for (uint8_t i = 0; i < SCH_MAX_TASKS; ++i) {
93          tasks[i].used = 0;
94          tasks[i].pTask = 0;
95      }
96      head = SCH_NONE;
97      next_seq = 0;
98      late_ticks = 0;
99      time_ticks = 0;
```

```
100     unlock(primask);
101 }
102 
103 void SCH_Update(void) {
104     ++time_ticks;
105     if (head == SCH_NONE) {
106         return;
107     }
108     if (tasks[head].delay > 0) {
109         --tasks[head].delay;
110     } else {
111         ++late_ticks;
112     }
113 }
114 
115 uint32_t SCH_Add_Task(void (*pFunction)(void), uint32_t DELAY,
    uint32_t PERIOD) {
116     if (pFunction == 0) {
117         return SCH_NO_TASK_ID;
118     }
119 
120     uint32_t primask = lock();
121     uint8_t slot = 0;
122     while (slot < SCH_MAX_TASKS && tasks[slot].used) {
123         ++slot;
124     }
125     if (slot == SCH_MAX_TASKS) {
126         unlock(primask);
127         return SCH_NO_TASK_ID;
128     }
129 
130     ++generation[slot];
131     tasks[slot].used = 1;
132     tasks[slot].pTask = pFunction;
133     tasks[slot].period = PERIOD / SCH_TICK_MS;
134     tasks[slot].id = ((uint32_t)generation[slot] << 8) | slot;
135     tasks[slot].seq = next_seq++;
136     insert(slot, DELAY / SCH_TICK_MS);
137 
138     uint32_t id = tasks[slot].id;
139     unlock(primask);
140     return id;
141 }
142 
143 uint8_t SCH_Delete_Task(uint32_t taskID) {
144     uint8_t slot = (uint8_t)(taskID & 0xFFu);
145     if (slot >= SCH_MAX_TASKS) {
146         return 0;
147     }
148 
149     uint32_t primask = lock();
150     if (!tasks[slot].used || tasks[slot].id != taskID) {
```

```
151         unlock(primask);
152         return 0;
153     }
154 
155     uint8_t prev = SCH_NONE;
156     uint8_t cur = head;
157     while (cur != slot) {
158         prev = cur;
159         cur = tasks[cur].next;
160     }
161 
162     uint8_t next = tasks[slot].next;
163     if (next != SCH_NONE) {
164         tasks[next].delay += tasks[slot].delay;
165     }
166     if (prev == SCH_NONE) {
167         head = next;
168     } else {
169         tasks[prev].next = next;
170     }
171     tasks[slot].used = 0;
172     tasks[slot].pTask = 0;
173     if (prev == SCH_NONE) {
174         apply_late_ticks();
175     }
176     unlock(primask);
177     return 1;
178 }
179 
180 void SCH_Dispatch_Tasks(void) {
181     for (;;) {
182         uint32_t primask = lock();
183         if (head == SCH_NONE || tasks[head].delay > 0) {
184             unlock(primask);
185             /* Sleep until the next interrupt. WFI runs with
    interrupts
186              * enabled: the Proteus CM3 model does not wake
    from WFI while
187              * PRIMASK is set. A tick landing just before WFI
    is picked up
188              * at the next HAL SysTick (1 ms), so tasks run < 1
     ms late. */
189             __WFI();
190             return;
191         }
192 
193         uint8_t slot = head;
194         void (*pTask)(void) = tasks[slot].pTask;
195         head = tasks[slot].next;
196 
197         if (tasks[slot].period > 0) {
198             insert(slot, tasks[slot].period);
```

```
199         } else {
200             tasks[slot].used = 0;
201             tasks[slot].pTask = 0;
202         }
203         apply_late_ticks();
204         unlock(primask);
205 
206         pTask();
207     }
208 }
209 
210 uint32_t SCH_Get_Time(void) {
211     return time_ticks * SCH_TICK_MS;
212 }
```

Program 4.3: scheduler.c

6 Application and demonstration code

The application (Program 4.4) registers:

- five periodic tasks `task_0_5s` ... `task_2_5s`; each one toggles its LED (T1–T5) and prints the current time and its name;
- `task_10ms`, a 10 ms timeout that prints `get_time` at every tick; its ID is kept in `tick_10ms_id`;
- `stop_10ms`, a one-shot task at 100 ms that deletes `task_10ms` by its ID, then deletes the same ID again to show that a stale ID is rejected;
- three one-shot tasks at 250, 1250 and 3330 ms that only print the time. They are added before the main loop with delays long enough that the loop is running before they expire.

All tasks are added with DELAY = PERIOD, so a periodic task first runs after one period. Each printed line has the form `[ time ms] name`. Sending one line takes about 2.2 ms at 115200 baud, well below the 10 ms tick. In total 10 of the 16 queue slots are used at start-up.

```
1  /* Print "[    1500 ms] T3 1.5s" so each callback shows when it
       really ran. */
2  static void log_task(const char *name) {
3      char buf[72];
4      int len = snprintf(buf, sizeof(buf), "[%8lu ms] %s\r\n",
5              (unsigned long)SCH_Get_Time(), name);
6      HAL_UART_Transmit(&huart1, (uint8_t *)buf, len, 10);
7  }
8  
9  /* Each task toggles its own LED, so LED_Tn blinks with half
       its task rate. */
10 static void task_0_5s(void) {
11     HAL_GPIO_TogglePin(LED_T1_GPIO_Port, LED_T1_Pin);
```

```
12      log_task("T1 0.5s");
13  }
14
15  static void task_1s(void) {
16      HAL_GPIO_TogglePin(LED_T2_GPIO_Port, LED_T2_Pin);
17      log_task("T2 1s");
18  }
19
20  static void task_1_5s(void) {
21      HAL_GPIO_TogglePin(LED_T3_GPIO_Port, LED_T3_Pin);
22      log_task("T3 1.5s");
23  }
24
25  static void task_2s(void) {
26      HAL_GPIO_TogglePin(LED_T4_GPIO_Port, LED_T4_Pin);
27      log_task("T4 2s");
28  }
29
30  static void task_2_5s(void) {
31      HAL_GPIO_TogglePin(LED_T5_GPIO_Port, LED_T5_Pin);
32      log_task("T5 2.5s");
33  }
34
35  /* Demo: a 10 ms timeout printing get_time, removed by ID after
        100 ms. */
36  static uint32_t tick_10ms_id = SCH_NO_TASK_ID;
37
38  static void task_10ms(void) {
39      log_task("tick 10ms");
40  }
41
42  static void stop_10ms(void) {
43      char msg[56];
44      uint8_t first = SCH_Delete_Task(tick_10ms_id);
45      uint8_t again = SCH_Delete_Task(tick_10ms_id);   /* stale
       ID -> 0 */
46      snprintf(msg, sizeof(msg), "delete ID 0x%04lX -> %u, again
       -> %u",
47               (unsigned long)tick_10ms_id, first, again);
48      log_task(msg);
49  }
50
51  /* Demo: one-shot timeouts set up before the main loop. */
52  static void once_250ms(void) {
53      log_task("one-shot 250 ms");
54  }
55
56  static void once_1250ms(void) {
57      log_task("one-shot 1250 ms");
58  }
59
60  static void once_3330ms(void) {
```

```
61      log_task("one-shot 3330 ms");
62  }
63
64  int main(void) {
65    /* ... HAL_Init, SystemClock_Config, MX_GPIO/TIM2/USART1_Init
        ... */
66    SCH_Init();
67    HAL_TIM_Base_Start_IT(&htim2);
68    /* DELAY = PERIOD: first run after one period, then every
       period (ms). */
69    SCH_Add_Task(task_0_5s, 500, 500);
70    SCH_Add_Task(task_1s, 1000, 1000);
71    SCH_Add_Task(task_1_5s, 1500, 1500);
72    SCH_Add_Task(task_2s, 2000, 2000);
73    SCH_Add_Task(task_2_5s, 2500, 2500);
74
75    /* 10 ms timeout: runs at 10, 20, ..., 100 ms. stop_10ms is
       due at 100 ms
76      * too but registered later, so the last tick prints before
       the delete. */
77    tick_10ms_id = SCH_Add_Task(task_10ms, 10, 10);
78    SCH_Add_Task(stop_10ms, 100, 0);
79
80    /* One-shots (PERIOD = 0); all fire long after the main loop
       starts. */
81    SCH_Add_Task(once_250ms, 250, 0);
82    SCH_Add_Task(once_1250ms, 1250, 0);
83    SCH_Add_Task(once_3330ms, 3330, 0);
84
85    log_task("Lab 4 scheduler demo start");
86    while (1) {
87      SCH_Dispatch_Tasks();
88    }
89  }
90
91  void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
92      if (htim->Instance == TIM2) {
93          SCH_Update();
94      }
95  }
```

Program 4.4: User code in main.c

7 Verification

7.1 Host unit tests

Before simulating, the scheduler was compiled with GCC on the PC. The CMSIS intrinsics such as `__disable_irq` and `__WFI` were replaced by empty functions, and `SCH_Update` and `SCH_Dispatch_Tasks` called alternately to simulate ticks. The tests

check that:

- in 15 s the five periodic tasks run exactly 30, 15, 10, 7 and 6 times, every run at a multiple of its period, with non-decreasing timestamps;
- tasks due at the same time run in registration order;
- a one-shot task frees its slot, a deleted ID returns 0 on a second delete, and an old ID does not delete a new task in the same slot;
- ticks that arrive while the main loop is busy (12 ticks without dispatch) are not lost;
- the 17th task is refused when the queue is full;
- deleting the head while it is counting down keeps the next task on time.

All tests pass. These tests check the logic only; the timing on the simulated MCU is verified below.

7.2 Simulation result

Figure 4.2 shows the Virtual Terminal from start-up until 6500 ms. The simulation was paused at 6.57 s (Figure 4.3), and the last line is 6500 ms, as expected.

```
[       0 ms] Lab 4 scheduler demo start
[      10 ms] tick 10ms
[      20 ms] tick 10ms
[      30 ms] tick 10ms
[      40 ms] tick 10ms
[      50 ms] tick 10ms
[      60 ms] tick 10ms
[      70 ms] tick 10ms
[      80 ms] tick 10ms
[      90 ms] tick 10ms
[     100 ms] tick 10ms
[     100 ms] delete ID 0x0105 -> 1, again -> 0
[     250 ms] one-shot 250 ms
[     500 ms] T1 0.5s
[    1000 ms] T1 0.5s
[    1000 ms] T2 1s
[    1250 ms] one-shot 1250 ms
[    1500 ms] T1 0.5s
[    1500 ms] T3 1.5s
[    2000 ms] T1 0.5s
[    2000 ms] T2 1s
[    2000 ms] T4 2s
[    2500 ms] T1 0.5s
[    2500 ms] T5 2.5s
[    3000 ms] T1 0.5s
[    3000 ms] T2 1s
[    3000 ms] T3 1.5s
[    3330 ms] one-shot 3330 ms
[    3500 ms] T1 0.5s
[    4000 ms] T1 0.5s
[    4000 ms] T2 1s
[    4000 ms] T4 2s
[    4500 ms] T1 0.5s
[    4500 ms] T3 1.5s
[    5000 ms] T1 0.5s
[    5000 ms] T2 1s
[    5000 ms] T5 2.5s
[    5500 ms] T1 0.5s
[    6000 ms] T1 0.5s
[    6000 ms] T2 1s
[    6000 ms] T3 1.5s
[    6000 ms] T4 2s
[    6500 ms] T1 0.5s
```

Hình 4.2: Virtual Terminal output from 0 to 6500 ms



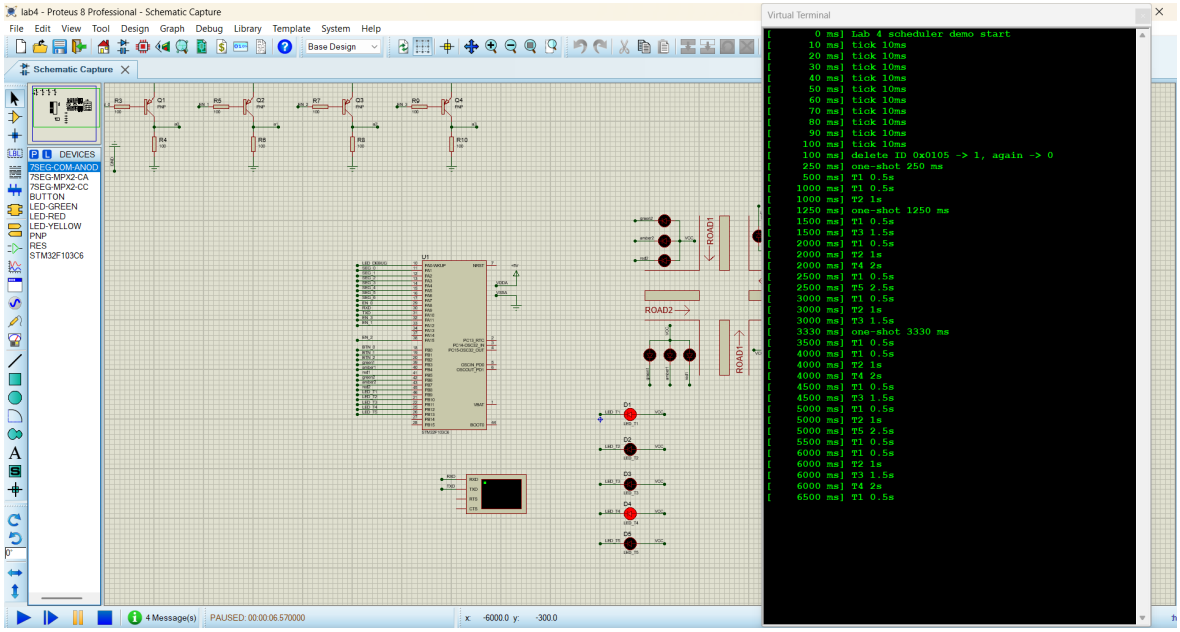
Hình 4.3: Simulation paused at 6.57 s

The output satisfies each item of the demonstration:

Requirement	Result
Regular 10 ms tick, callback every 10 ms printing `get_time`	`tick 10ms` at 10, 20, . . . , 100 ms
Timestamps accurate to 10 ms	Every line is an exact multiple of its period or delay
Several timeouts at the same time	Five periodic tasks, the 10 ms task and the one-shot tasks run together
One-shot tasks set up before the main loop	Lines at 250, 1250 and 3330 ms, one line each
Callbacks in the right order	Same-time tasks print in registration order, e.g. 6000 ms: T1, T2, T3, T4
`SCH_Delete_Task` by ID	`delete ID 0x0105 -> 1, again -> 0` at 100 ms; no `tick 10ms` after 100 ms

Bảng 4.3: Demonstration requirements and results

The ID `0x0105` is slot 5 (the sixth task added) with generation 1. The LEDs agree with the log: each LED is toggled once per run of its task, starting from off, so at 6.57 s T1 (13 toggles) and T4 (3 toggles) are on and the others are off, which is what the simulation shows. Figure 4.4 shows the full simulation window.



Hình 4.4: Simulation at 6.57 s: D1 and D4 on, terminal output on the right

8 Problems found during simulation

8.1 Firmware stopped in the first version

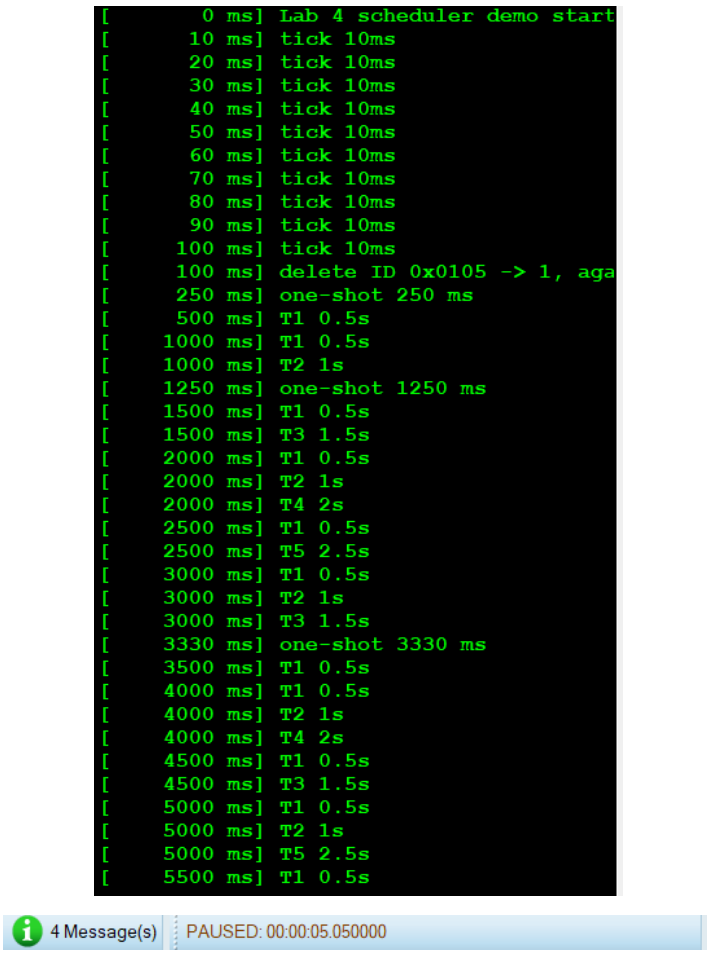
In the first version, `SCH_Dispatch_Tasks` executed `__WFI()` while interrupts were still disabled. On a real Cortex-M3 this is allowed: a pending interrupt wakes the core even when PRIMASK masks it. In Proteus, however, nothing was printed and no LED changed. After moving `__WFI()` after interrupts are enabled again, the program ran normally, so the Proteus CM3 model most likely does not wake from WFI while PRIMASK is set. The cost of the new order is small: a TIM2 tick arriving just before `__WFI()` is handled at the next interrupt, and the HAL SysTick interrupt occurs every 1 ms, so a task can be delayed by less than 1 ms.

8.2 Timer period in Proteus

The first configuration of TIM2 was PSC 7999 / ARR 9 ($8000 \times 10/8\,\text{MHz} = 10\,\text{ms}$). With it, the time printed by the firmware ran ahead of the simulation time. Pausing the simulation gave the results in Table 4.4.

Proteus time	Last line printed	Proteus time × 10/9	Expected (9 ms tick)
0.68 s	500 ms	0.76 s	500 ms
5.05 s	5500 ms	5.61 s	5500 ms
9.39 s	10000 ms	10.43 s	10000 ms
21.96 s	24000 ms	24.40 s	24000 ms

Bảng 4.4: Measurements with PSC 7999 / ARR 9



Hình 4.5: PSC 7999 / ARR 9: the firmware has already printed 5500 ms when the simulation is paused at 5.05 s

All four measurements match a tick of 9 ms instead of 10 ms. The UART output stayed correct, so the system clock itself was not wrong. On a real STM32 the update period is $(PSC+1)(ARR+1)$ timer clocks; the results match a timer model that counts ARR instead of $ARR+1$ steps per period. This conclusion comes from the measurements above, not from Proteus documentation.

Missing one step per period gives a relative error of $1/(ARR+1)$, so the error can be made small by putting most of the division into ARR. With PSC 79 / ARR 999 the period on a real MCU is still exactly 10 ms, and if the model counts 999 steps instead of 1000 the error is only 0.1 %. After the change, the simulation paused at 6.57 s shows 6500 ms as the last line (Figure 4.3), so the 11 % error is gone. This short run does not measure whether a residual error of about 1 % remains.

8.3 Random Proteus exception

Once, Proteus stopped with “Internal Exception: access violation in module UNKNOWN” at 0.508 s. Running the same HEX file again worked without error, so this was a problem of the simulator, not of the firmware.

9 Conclusion

A cooperative scheduler was implemented with a delta queue, so `SCH_Update` runs in O(1) inside the 10 ms timer interrupt. Tasks are added and deleted by ID, any number up to 16 can be registered at the same time, periodic and one-shot tasks are supported, and tasks due at the same time run in registration order. The demonstration in Proteus shows the five periodic tasks, a 10 ms timeout printing the time, deletion by ID and one-shot tasks, with timestamps accurate to 10 ms. Two Proteus-specific issues were found and handled: WFI must be executed with interrupts enabled, and the timer should use a large ARR value.

The source code is available at:
https://github.com/tienanbui1708/Microcontroller-Lab